C#

```
public string ImageTitle
{
    get => String.IsNullOrEmpty(ImageProperties.Title) ? ImageName :
ImageProperties.Title;
    set
    {
        if (ImageProperties.Title != value)
        {
            ImageProperties.Title = value;
            var ignoreResult = ImageProperties.SavePropertiesAsync();
            OnPropertyChanged();
        }
    }
}
```

As you can see, the setter updates the `ImageProperties.Title` property and then calls `SavePropertiesAsync` to write the new value to the file. (This is an async method, but we can't use the `await` keyword in a property - and you wouldn't want to because property getters and setters should complete immediately. So instead, you would call the method and ignore the `Task` object it returns.)

Going further

Now that you've completed this lab, you have enough binding knowledge tackle a problem on your own.

As you might have noticed, if you change the zoom level on the details page, it resets automatically when you navigate backward then select the same image again. Can you figure out how to preserve and restore the zoom level for each image individually? Good luck!

You should have all the info you need in this tutorial, but if you need more guidance, the data binding docs are only a click away. Start here:

- {x:Bind} markup extension
- Data binding in depth

Sample data on the design surface, and for prototyping

Article • 10/20/2022

> **Note**
>
> The degree to which you need sample data—and how much it will help you—depends on whether your bindings use the **{Binding} markup extension** or the **{x:Bind} markup extension**. The techniques described in this topic are based on the use of a **DataContext**, so they're only appropriate for **{Binding}**. But if you're using **{x:Bind}** then your bindings at least show placeholder values on the design surface (even for items controls), so you don't have quite the same need for sample data.

It may be impossible or undesirable (perhaps for reasons of privacy or performance) for your app to display live data on the design surface in Microsoft Visual Studio or Blend for Visual Studio. In order to have your controls populated with data (so that you can work on your app's layout, templates, and other visual properties), there are various ways in which you can use design-time sample data. Sample data can also be really useful and time-saving if you're building a sketch (or prototype) app. You can use sample data in your sketch or prototype at run-time to illustrate your ideas without going as far as connecting to real, live data.

Sample apps that demonstrate {Binding}

- Download the Bookstore1 app.
- Download the Bookstore2 app.

> **Note**
>
> Screenshots in this article were taken from a previous version of Visual Studio. They might not precisely match your development experience if you are using Visual Studio 2019.

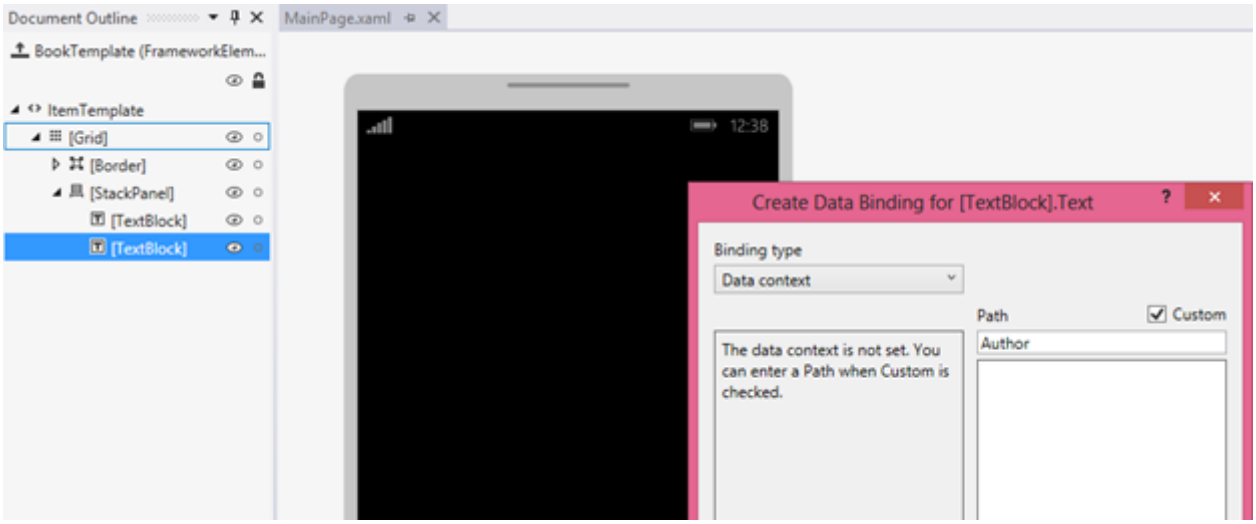
Setting DataContext in markup

It's a fairly common developer practice to use imperative code (in code-behind) to set a page or user control's **DataContext** to a view model instance.

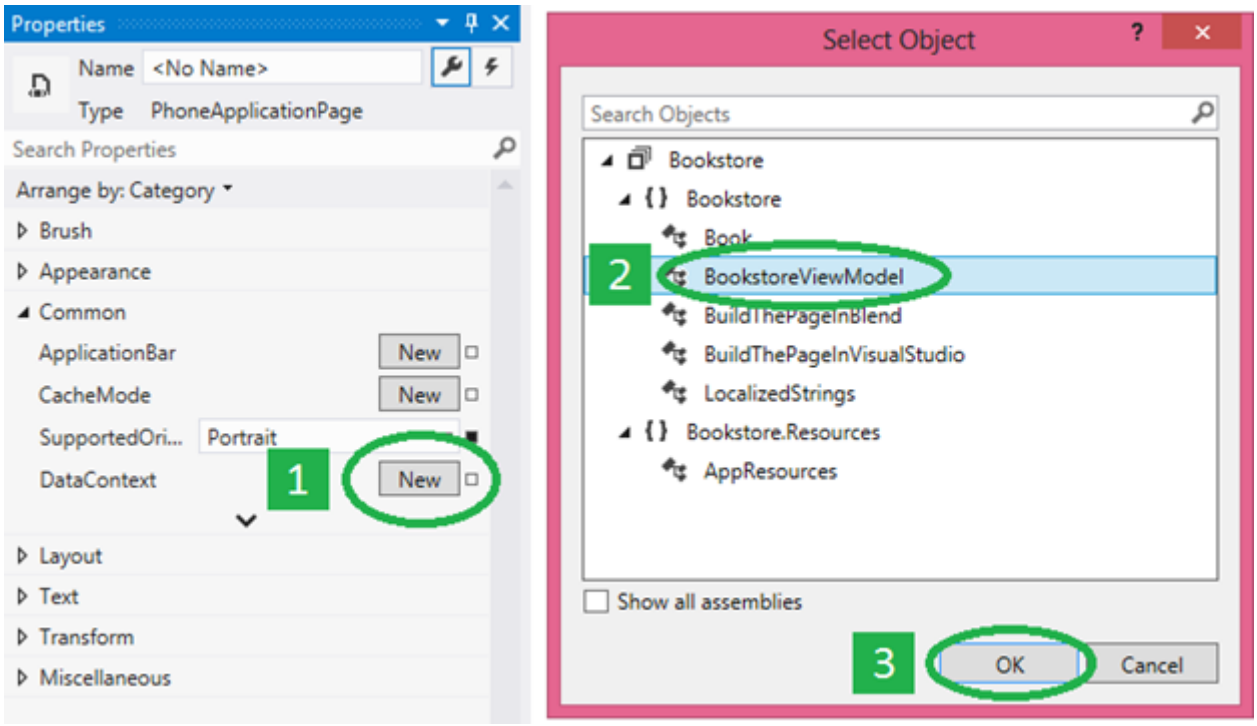
C#

```
public MainPage()
{
    InitializeComponent();
    this.DataContext = new BookstoreViewModel();
}
```

But if you do that then your page isn't as "designable" as it could be. The reason is that when your XAML page is opened in Visual Studio or Blend for Visual Studio, the imperative code that assigns the **DataContext** value is never run (in fact, none of your code-behind is executed). The XAML tools do of course parse your markup and instantiate any objects declared in it, but they don't actually instantiate your page's type itself. The result is that you won't see any data in your controls or in the **Create Data Binding** dialog, and your page will be more challenging to style and to lay out.



The first remedy to try is to comment out that **DataContext** assignment and set the **DataContext** in your page markup instead. That way, your live data shows up at design-time as well as at run-time. To do this, first open your XAML page. Then, in the **Document Outline** window, click the root designable element (usually with the label **[Page]**) to select it. In the **Properties** window, find the **DataContext** property (inside the Common category), and modify it. Select your view model type from the **Select Object** dialog box, and then click **OK**.

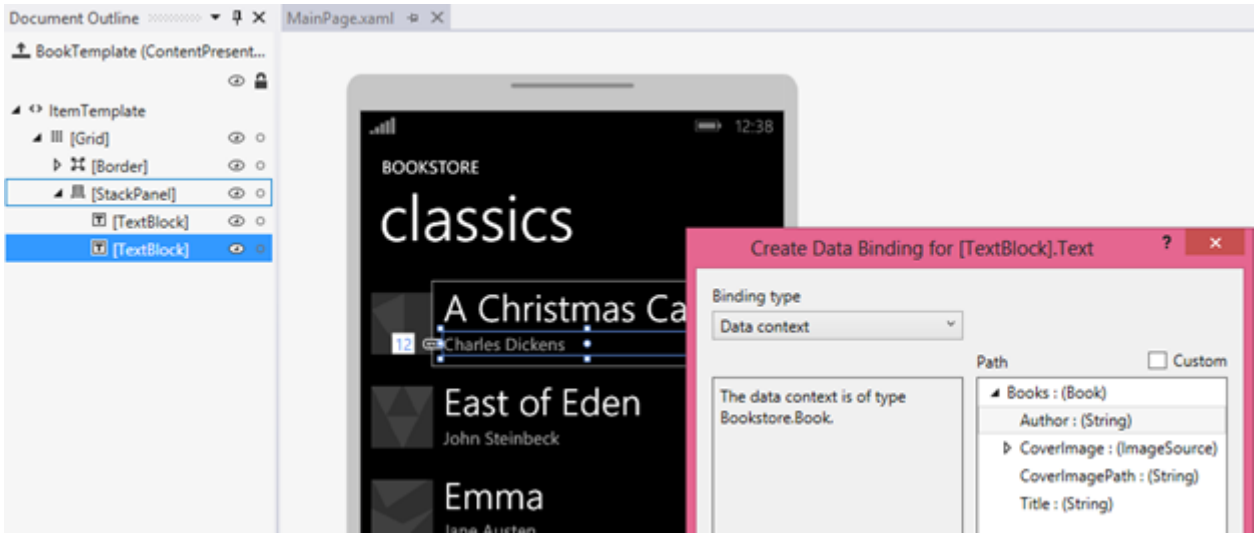


Here's what the resulting markup looks like.

XAML

```
<Page ... >
    <Page.DataContext>
        <local:BookstoreViewModel/>
    </Page.DataContext>
```

And here's what the design surface looks like now that your bindings can resolve. Notice that the **Path** picker in the **Create Data Binding** dialog is now populated, based on the **DataContext** type and the properties that you can bind to.



The **Create Data Binding** dialog only needs a type to work from, but the bindings need the properties to be initialized with values. If you don't want to reach out to your cloud service at design-time (due to performance, paying for data transfer, privacy issues, that

kind of thing) then your initialization code can check to see whether your app is running in a design tool (such as Visual Studio or Blend for Visual Studio) and in that case load sample data for use at design-time only.

```
C#
```

```
if (Windows.ApplicationModel.DesignMode.DesignModeEnabled)
{
    // Load design-time books.
}
else
{
    // Load books from a cloud service.
}
```

You could use a view model locator if you need to pass parameters to your initialization code. A view model locator is a class that you can put into app resources. It has a property that exposes the view model, and your page's **DataContext** binds to that property. Another pattern that the locator or the view model can use is dependency injection, which can construct a design-time or a run-time data provider (each of which implements a common interface), as applicable.

"Sample data from class" and design-time attributes

If for whatever reason none of the options in the previous section work for you then you still have plenty of design-time data options available via XAML tools features and design-time attributes. One good option is the **Create Sample Data from Class** feature in Blend for Visual Studio. You can find that command on one of the buttons at the top of the **Data** panel.

All you need to do is to specify a class for the command to use. The command then does two important things for you. First, it generates a XAML file that contains sample data suitable for hydrating an instance of your chosen class and all of its members, recursively (in fact, the tooling works equally well with XAML or JSON files). Second, it populates the **Data** panel with the schema of your chosen class. You can then drag members from the **Data** panel onto the design surface to perform various tasks. Depending on what you drag and where you drop it, you can add bindings to existing controls (using **{Binding}**), or create new controls and bind them at the same time. In either case, the operation also sets a design-time data context (**d:DataContext**) for you (if one is not already set) on the layout root of your page. That design-time data context uses the **d:DesignData** attribute to get its sample data from the XAML file that was

generated (which, by the way, you are free to find in your project and edit so that it contains the sample data you want).

XAML

```
<Page ...
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    mc:Ignorable="d">
    <Grid ... d:DataContext="{d:DesignData
/SampleData/RecordingViewModelSampleData.xaml}"/>
        <ListView ItemsSource="{Binding Recordings}" ... />
        ...
    </Grid>
</Page>
```

The various xmlns declarations mean that attributes with the **d:** prefix are interpreted only at design-time and are ignored at run-time. So the **d:DataContext** attribute only affects the value of the DataContext property at design-time; it has no effect at run-time. You can even set both **d:DataContext** and **DataContext** in markup if you like. **d:DataContext** will override at design-time, and **DataContext** will override at run-time. These same override rules apply to all design-time and run-time attributes.

The **d:DataContext** attribute, and all other design-time attributes, are documented in the Design-Time Attributes topic, which is still valid for Universal Windows Platform (UWP) apps.

CollectionViewSource doesn't have a **DataContext** property, but it does have a **Source** property. Consequently, there's a **d:Source** property that you can use to set design-time-only sample data on a **CollectionViewSource**.

XAML

```
    <Page.Resources>
        <CollectionViewSource x:Name="RecordingsCollection" Source="{Binding
Recordings}"
            d:Source="{d:DesignData
/SampleData/RecordingsSampleData.xaml}"/>
    </Page.Resources>

    ...

        <ListView ItemsSource="{Binding Source={StaticResource
RecordingsCollection}}" ... />
    ...
```

For this to work, you would have a class named `Recordings : ObservableCollection<Recording>`, and you would edit the sample data XAML file so that it contains only a **Recordings** object (with **Recording** objects inside that), as shown here.

XML

```
<Quickstart:Recordings xmlns:Quickstart="using:Quickstart">
    <Quickstart:Recording ArtistName="Mollis massa" CompositionName="Cubilia
metus"
        OneLineSummary="Morbi adipiscing sed" ReleaseDateTime="01/01/1800
15:53:17"/>
    <Quickstart:Recording ArtistName="Vulputate nunc"
CompositionName="Parturient vestibulum"
        OneLineSummary="Dapibus praesent netus amet vestibulum"
ReleaseDateTime="01/01/1800 15:53:17"/>
    <Quickstart:Recording ArtistName="Phasellus accumsan"
CompositionName="Sit bibendum"
        OneLineSummary="Vestibulum egestas montes dictumst"
ReleaseDateTime="01/01/1800 15:53:17"/>
</Quickstart:Recordings>
```

If you use a JSON sample data file instead of XAML, you must set the **Type** property.

XAML

```
    d:Source="{d:DesignData /SampleData/RecordingsSampleData.json,
Type=local:Recordings}"
```

So far, we've been using **d:DesignData** to load design-time sample data from a XAML or JSON file. An alternative to that is the **d:DesignInstance** markup extension, which indicates that the design-time source is based on the class specified by the **Type** property. Here's an example.

XAML

```
    <CollectionViewSource x:Name="RecordingsCollection" Source="{Binding
Recordings}"
        d:Source="{d:DesignInstance Type=local:Recordings,
IsDesignTimeCreatable=True}"/>
```

The **IsDesignTimeCreatable** property indicates that the design tool should actually create an instance of the class, which implies that the class has a public default constructor, and that it populates itself with data (either real or sample). If you don't set **IsDesignTimeCreatable** (or if you set it to **False**) then you won't get sample data displayed on the design surface. All the design tool does in that case is to parse the

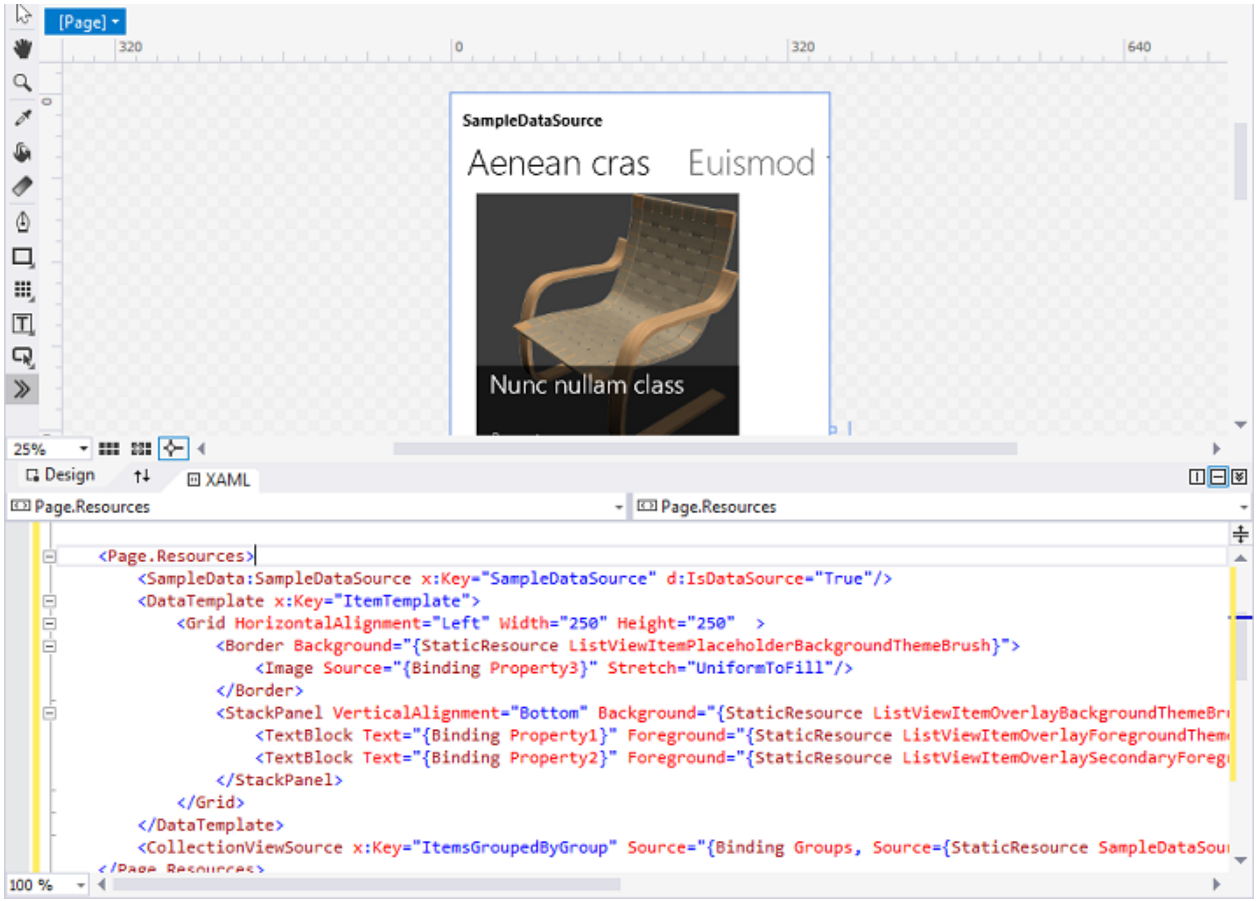
class for its bindable properties and display these in the **Data** panel and in the **Create Data Binding** dialog.

Sample data for prototyping

For prototyping, you want sample data at both design-time and at run-time. For that use case, Blend for Visual Studio has the **New Sample Data** feature. You can find that command on one of the buttons at the top of the **Data** panel.

Instead of specifying a class, you can actually design the schema of your sample data source right in the **Data** panel. You can also edit sample data values in the **Data** panel: there's no need to open and edit a file (although, you can still do that if you prefer).

The **New Sample Data** feature uses DataContext, and not **d:DataContext**, so that the sample data is available when you run your sketch or prototype as well as while you're designing it. And the **Data** panel really speeds up your designing and binding tasks. For example, simply dragging a collection property from the **Data** panel onto the design surface generates a data-bound items control and the necessary templates, all ready to build and run.



Bind hierarchical data and create a master/details view

Article • 10/20/2022

> **Note** Also see the Master/detail sample.

You can make a multi-level master/details (also known as list-details) view of hierarchical data by binding items controls to **CollectionViewSource** instances that are bound together in a chain. In this topic we use the {x:Bind} markup extension where possible, and the more flexible (but less performant) {Binding} markup extension where necessary.

One common structure for Universal Windows Platform (UWP) apps is to navigate to different details pages when a user makes a selection in a master list. This is useful when you want to provide a rich visual representation of each item at every level in a hierarchy. Another option is to display multiple levels of data on a single page. This is useful when you want to display a few simple lists that let the user quickly drill down to an item of interest. This topic describes how to implement this interaction. The **CollectionViewSource** instances keep track of the current selection at each hierarchical level.

We'll create a view of a sports team hierarchy that is organized into lists for leagues, divisions, and teams, and includes a team details view. When you select an item from any list, the subsequent views update automatically.

Prerequisites

This topic assumes that you know how to create a basic UWP app. For instructions on creating your first UWP app, see Create your first UWP app using C# or Visual Basic.

Create the project

Create a new **Blank Application (Windows Universal)** project. Name it "MasterDetailsBinding".

Create the data model

Add a new class to your project, name it ViewModel.cs, and add this code to it. This will be your binding source class.

C#

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace MasterDetailsBinding
{
    public class Team
    {
        public string Name { get; set; }
        public int Wins { get; set; }
        public int Losses { get; set; }
    }

    public class Division
    {
        public string Name { get; set; }
        public IEnumerable<Team> Teams { get; set; }
    }

    public class League
    {
        public string Name { get; set; }
        public IEnumerable<Division> Divisions { get; set; }
    }

    public class LeagueList : List<League>
    {
        public LeagueList()
        {
```

```
            this.AddRange(GetLeague().ToList());
        }

        public IEnumerable<League> GetLeague()
        {
            return from x in Enumerable.Range(1, 2)
                   select new League
                   {
                       Name = "League " + x,
                       Divisions = GetDivisions(x).ToList()
                   };
        }

        public IEnumerable<Division> GetDivisions(int x)
        {
            return from y in Enumerable.Range(1, 3)
                   select new Division
                   {
                       Name = String.Format("Division {0}-{1}", x, y),
                       Teams = GetTeams(x, y).ToList()
                   };
        }

        public IEnumerable<Team> GetTeams(int x, int y)
        {
            return from z in Enumerable.Range(1, 4)
                   select new Team
                   {
                       Name = String.Format("Team {0}-{1}-{2}", x, y, z),
                       Wins = 25 - (x * y * z),
                       Losses = x * y * z
                   };
        }
    }
}
```

Create the view

Next, expose the binding source class from the class that represents your page of markup. We do that by adding a property of type **LeagueList** to **MainPage**.

C#

```
namespace MasterDetailsBinding
{
    /// <summary>
    /// An empty page that can be used on its own or navigated to within a
Frame.
    /// </summary>
    public sealed partial class MainPage : Page
    {
```

```
        public MainPage()
        {
            this.InitializeComponent();
            this.ViewModel = new LeagueList();
        }
        public LeagueList ViewModel { get; set; }
    }
}
```

Finally, replace the contents of the MainPage.xaml file with the following markup, which declares three **CollectionViewSource** instances and binds them together in a chain. The subsequent controls can then bind to the appropriate **CollectionViewSource**, depending on its level in the hierarchy.

XML

```
<Page
    x:Class="MasterDetailsBinding.MainPage"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="using:MasterDetailsBinding"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    mc:Ignorable="d">

    <Page.Resources>
        <CollectionViewSource x:Name="Leagues"
            Source="{x:Bind ViewModel}"/>
        <CollectionViewSource x:Name="Divisions"
            Source="{Binding Divisions, Source={StaticResource Leagues}}"/>
        <CollectionViewSource x:Name="Teams"
            Source="{Binding Teams, Source={StaticResource Divisions}}"/>

        <Style TargetType="TextBlock">
            <Setter Property="FontSize" Value="15"/>
            <Setter Property="FontWeight" Value="Bold"/>
        </Style>

        <Style TargetType="ListBox">
            <Setter Property="FontSize" Value="15"/>
        </Style>

        <Style TargetType="ContentControl">
            <Setter Property="FontSize" Value="15"/>
        </Style>

    </Page.Resources>

    <Grid Background="{ThemeResource ApplicationPageBackgroundThemeBrush}">

        <StackPanel Orientation="Horizontal">
```

```
            <!-- All Leagues view -->

            <StackPanel Margin="5">
                <TextBlock Text="All Leagues"/>
                <ListBox ItemsSource="{Binding Source={StaticResource
Leagues}}"
                    DisplayMemberPath="Name"/>
            </StackPanel>

            <!-- League/Divisions view -->

            <StackPanel Margin="5">
                <TextBlock Text="{Binding Name, Source={StaticResource
Leagues}}"/>
                <ListBox ItemsSource="{Binding Source={StaticResource
Divisions}}"
                    DisplayMemberPath="Name"/>
            </StackPanel>

            <!-- Division/Teams view -->

            <StackPanel Margin="5">
                <TextBlock Text="{Binding Name, Source={StaticResource
Divisions}}"/>
                <ListBox ItemsSource="{Binding Source={StaticResource
Teams}}"
                    DisplayMemberPath="Name"/>
            </StackPanel>

            <!-- Team view -->

            <ContentControl Content="{Binding Source={StaticResource
Teams}}">
                <ContentControl.ContentTemplate>
                    <DataTemplate>
                        <StackPanel Margin="5">
                            <TextBlock Text="{Binding Name}"
                                FontSize="15" FontWeight="Bold"/>
                            <StackPanel Orientation="Horizontal"
Margin="10,10">
                                <TextBlock Text="Wins:" Margin="0,0,5,0"/>
                                <TextBlock Text="{Binding Wins}"/>
                            </StackPanel>
                            <StackPanel Orientation="Horizontal"
Margin="10,0">
                                <TextBlock Text="Losses:" Margin="0,0,5,0"/>
                                <TextBlock Text="{Binding Losses}"/>
                            </StackPanel>
                        </StackPanel>
                    </DataTemplate>
                </ContentControl.ContentTemplate>
            </ContentControl>

        </StackPanel>
```

```
    </Grid>
</Page>
```

Note that by binding directly to the **CollectionViewSource**, you're implying that you want to bind to the current item in bindings where the path cannot be found on the collection itself. There's no need to specify the **CurrentItem** property as the path for the binding, although you can do that if there's any ambiguity. For example, the **ContentControl** representing the team view has its **Content** property bound to the `Teams` **CollectionViewSource**. However, the controls in the **DataTemplate** bind to properties of the `Team` class because the **CollectionViewSource** automatically supplies the currently selected team from the teams list when necessary.

Data binding and MVVM

Article • 10/20/2022

Model-View-ViewModel (MVVM) is a UI architectural design pattern for decoupling UI and non-UI code. With MVVM, you define your UI declaratively in XAML and use data binding markup to link it to other layers containing data and commands. The data binding infrastructure provides a loose coupling that keeps the UI and the linked data synchronized and routes user input to the appropriate commands.

Because it provides loose coupling, the use of data binding reduces hard dependencies between different kinds of code. This makes it easier to change individual code units (methods, classes, controls, etc.) without causing unintended side effects in other units. This decoupling is an example of the *separation of concerns*, which is an important concept in many design patterns.

Benefits of MVVM

Decoupling your code has many benefits, including:

- Enabling an iterative, exploratory coding style. Change that is isolated is less risky and easier to experiment with.
- Simplifying unit testing. Code units that are isolated from one another can be tested individually and outside of production environments.
- Supporting team collaboration. Decoupled code that adheres to well-designed interfaces can be developed by separate individuals or teams, and integrated later.
- Improving maintainability. Fixing bugs in decoupled code is less likely to cause regressions in other code.

In contrast with MVVM, an app with a more conventional "code-behind" structure typically uses data binding for display-only data, and responds to user input by directly handling events exposed by controls. The event handlers are implemented in code-behind files (such as MainPage.xaml.cs), and are often tightly coupled to the controls, typically containing code that manipulates the UI directly. This makes it difficult or impossible to replace a control without having to update the event handling code. With this architecture, code-behind files often accumulate code that isn't directly related to the UI, such as database-access code, which ends up being duplicated and modified for use with other pages.

App layers

When using the MVVM pattern, an app is divided into the following layers:

- The **model** layer defines the types that represent your business data. This includes everything required to model the core app domain, and often includes core app logic. This layer is completely independent of the view and view-model layers, and often resides partially in the cloud. Given a fully implemented model layer, you can create multiple different client apps if you so choose, such as UWP and web apps that work with the same underlying data.
- The **view** layer defines the UI using XAML markup. The markup includes data binding expressions (such as x:Bind) that define the connection between specific UI components and various view-model and model members. Code-behind files are sometimes used as part of the view layer to contain additional code needed to customize or manipulate the UI, or to extract data from event handler arguments before calling a view-model method that performs the work.
- The **view-model** layer provides data binding targets for the view. In many cases, the view-model exposes the model directly, or provides members that wrap specific model members. The view-model can also define members for keeping track of data that is relevant to the UI but not to the model, such as the display order of a list of items. The view-model also serves as an integration point with other services such as database-access code. For simple projects, you might not need a separate model layer, but only a view-model that encapsulates all the data you need.

Basic and advanced MVVM

As with any design pattern, there is more than one way to implement MVVM, and many different techniques are considered part of MVVM. For this reason, there are several different third-party MVVM frameworks supporting the various XAML-based platforms, including UWP. However, these frameworks generally include multiple services for implementing decoupled architecture, making the exact definition of MVVM somewhat ambiguous.

Although sophisticated MVVM frameworks can be very useful, especially for enterprise-scale projects, there is typically a cost associated with adopting any particular pattern or technique, and the benefits are not always clear, depending on the scale and size of your project. Fortunately, you can adopt only those techniques that provide a clear and tangible benefit, and ignore others until you need them.

In particular, you can get a lot of benefit simply by understanding and applying the full power of data binding and separating your app logic into the layers described earlier. This can be achieved using only the capabilities provided by the Windows SDK, and

without using any external frameworks. In particular, the {x:Bind} markup extension makes data binding easier and higher performing than in previous XAML platforms, eliminating the need for a lot of the boilerplate code required earlier.

For additional guidance on using basic, out-of-the-box MVVM, check out the Customers Orders Database sample on GitHub. Many of the other UWP app samples also use a basic MVVM architecture, and the Traffic App sample includes both code-behind and MVVM versions, with notes describing the MVVM conversion.

See also

Topics

Data binding in depth
{x:Bind} markup extension

Samples

Customers Orders Database sample
VanArsdel Inventory sample
Traffic App sample

Files, folders, and libraries

Article • 06/29/2023

You use the APIs in the Windows.Storage, Windows.Storage.Streams, and Windows.Storage.Pickers namespaces to read and write text and other data formats in files, and to manage files and folders. In this section, you'll also learn about reading and writing app settings, about file and folder pickers, and about special sand-boxed locations such as the Video/Music library.

Topic	Description
Enumerate and query files and folders	Access files and folders in either a folder, library, device, or network location. You can also query the files and folders in a location by constructing file and folder queries.
Create, write, and read a file	Read and write a file using a StorageFile object.
Best practices for writing to files	Learn best practices for using various file writing methods of the FileIO and PathIO classes.
Get file properties	Get properties—top-level, basic, and extended—for a file represented by a StorageFile object.
Open files and folders with a picker	Access files and folders by letting the user interact with a picker. You can use the FolderPicker to gain access to a folder.
Save a file with a picker	Use FileSavePicker to let users specify the name and location where they want your app to save a file.
Accessing HomeGroup content	Access content stored in the user's HomeGroup folder, including pictures, music, and videos.
Determining availability of Microsoft OneDrive files	Determine if a Microsoft OneDrive file is available using the StorageFile.IsAvailable property.

Topic	Description
Files and folders in the Music, Pictures, and Videos libraries	Add existing folders of music, pictures, or videos to the corresponding libraries. You can also remove folders from libraries, get the list of folders in a library, and discover stored photos, music, and videos.
Track recently used files and folders	Track files that your user accesses frequently by adding them to your app's most recently used list (MRU). The platform manages the MRU for you by sorting items based on when they were last accessed, and by removing the oldest item when the list's 25-item limit is reached. All apps have their own MRU.
Track file system changes in the background	Track changes to the file system, even when the app isn't running.
Access the SD card	You can store and access non-essential data on an optional microSD card, especially on low-cost mobile devices that have limited internal storage.
File access permissions	Apps can access certain file system locations by default. Apps can also access additional locations through the file picker, or by declaring capabilities.
Fast access to file properties in UWP	Efficiently gather a list of files and their properties from a library to use in a UWP app.

Related samples

Folder enumeration sample

File access sample

File picker sample

Enumerate and query files and folders

Article • 10/20/2022

Access files and folders in either a folder, library, device, or network location. You can also query the files and folders in a location by constructing file and folder queries.

For guidance on how to store your Universal Windows Platform app's data, see the ApplicationData class.

> **Note**
>
> For a complete sample, see the **Folder enumeration sample**.

Prerequisites

- **Understand async programming for Universal Windows Platform (UWP) apps**

  You can learn how to write asynchronous apps in C# or Visual Basic, see Call asynchronous APIs in C# or Visual Basic. To learn how to write asynchronous apps in C++/WinRT, see Concurrency and asynchronous operations with C++/WinRT. To learn how to write asynchronous apps in C++/CX, see Asynchronous programming in C++/CX.

- **Access permissions to the location**

  For example, the code in these examples require the **picturesLibrary** capability, but your location may require a different capability or no capability at all. To learn more, see File access permissions.

Enumerate files and folders in a location

> **Note**
>
> Remember to declare the **picturesLibrary** capability.

In this example we first use the **StorageFolder.GetFilesAsync** method to get all the files in the root folder of the **KnownFolders.PicturesLibrary** (not in subfolders) and list the name of each file. Next, we use the **StorageFolder.GetFoldersAsync** method to get all the subfolders in the **PicturesLibrary** and list the name of each subfolder.

C#

```
StorageFolder picturesFolder = KnownFolders.PicturesLibrary;
StringBuilder outputText = new StringBuilder();

IReadOnlyList<StorageFile> fileList = await picturesFolder.GetFilesAsync();

outputText.AppendLine("Files:");
foreach (StorageFile file in fileList)
{
    outputText.Append(file.Name + "\n");
}

IReadOnlyList<StorageFolder> folderList = await
picturesFolder.GetFoldersAsync();

outputText.AppendLine("Folders:");
foreach (StorageFolder folder in folderList)
{
    outputText.Append(folder.DisplayName + "\n");
}
```

> **Note**
>
> In C# or Visual Basic, remember to put the **async** keyword in the method declaration of any method in which you use the **await** operator.

Alternatively, you can use the StorageFolder.GetItemsAsync method to get all items (both files and subfolders) in a particular location. The following example uses the **GetItemsAsync** method to get all files and subfolders in the root folder of the KnownFolders.PicturesLibrary (not in subfolders). Then the example lists the name of each file and subfolder. If the item is a subfolder, the example appends `"folder"` to the name.

C#

```
StorageFolder picturesFolder = KnownFolders.PicturesLibrary;
StringBuilder outputText = new StringBuilder();

IReadOnlyList<IStorageItem> itemsList = await
picturesFolder.GetItemsAsync();

foreach (var item in itemsList)
{
    if (item is StorageFolder)
    {
        outputText.Append(item.Name + " folder\n");
```

```
    }
    else
    {
        outputText.Append(item.Name + "\n");
    }
}
```

Query files in a location and enumerate matching files

In this example we query for all the files in the KnownFolders.PicturesLibrary grouped by the month, and this time the example recurses into subfolders. First, we call StorageFolder.CreateFolderQuery and pass the CommonFolderQuery.GroupByMonth value to the method. That gives us a StorageFolderQueryResult object.

Next we call StorageFolderQueryResult.GetFoldersAsync which returns StorageFolder objects representing virtual folders. In this case we're grouping by month, so the virtual folders each represent a group of files with the same month.

C#

```
StorageFolder picturesFolder = KnownFolders.PicturesLibrary;

StorageFolderQueryResult queryResult =
    picturesFolder.CreateFolderQuery(CommonFolderQuery.GroupByMonth);

IReadOnlyList<StorageFolder> folderList =
    await queryResult.GetFoldersAsync();

StringBuilder outputText = new StringBuilder();

foreach (StorageFolder folder in folderList)
{
    IReadOnlyList<StorageFile> fileList = await folder.GetFilesAsync();

    // Print the month and number of files in this group.
    outputText.AppendLine(folder.Name + " (" + fileList.Count + ")");

    foreach (StorageFile file in fileList)
    {
        // Print the name of the file.
        outputText.AppendLine("    " + file.Name);
    }
}
```

The output of the example looks similar to the following.

syntax

```
July 2015 (2)
   MyImage3.png
   MyImage4.png
December 2014 (2)
   MyImage1.png
   MyImage2.png
```

Create, write, and read a file

Article • 10/20/2022

Important APIs

- StorageFolder class
- StorageFile class
- FileIO class

Read and write a file using a StorageFile object.

> **Note**
>
> For a complete sample, see the File access sample.

Prerequisites

- **Understand async programming for Universal Windows Platform (UWP) apps**

  You can learn how to write asynchronous apps in C# or Visual Basic, see Call asynchronous APIs in C# or Visual Basic. To learn how to write asynchronous apps in C++/WinRT, see Concurrency and asynchronous operations with C++/WinRT. To learn how to write asynchronous apps in C++/CX, see Asynchronous programming in C++/CX.

- **Know how to get the file that you want to read from, write to, or both**

  You can learn how to get a file by using a file picker in Open files and folders with a picker.

Creating a file

Here's how to create a file in the app's local folder. If it already exists, we replace it.

C#

```
// Create sample file; replace if exists.
Windows.Storage.StorageFolder storageFolder =
    Windows.Storage.ApplicationData.Current.LocalFolder;
Windows.Storage.StorageFile sampleFile =
    await storageFolder.CreateFileAsync("sample.txt",
        Windows.Storage.CreationCollisionOption.ReplaceExisting);
```

Writing to a file

Here's how to write to a writable file on disk using the StorageFile class. The common first step for each of the ways of writing to a file (unless you're writing to the file immediately after creating it) is to get the file with StorageFolder.GetFileAsync.

```
C#

Windows.Storage.StorageFolder storageFolder =
    Windows.Storage.ApplicationData.Current.LocalFolder;
Windows.Storage.StorageFile sampleFile =
    await storageFolder.GetFileAsync("sample.txt");
```

Writing text to a file

Write text to your file by calling the FileIO.WriteTextAsync method.

```
C#

await Windows.Storage.FileIO.WriteTextAsync(sampleFile, "Swift as a
shadow");
```

Writing bytes to a file by using a buffer (2 steps)

1. First, call CryptographicBuffer.ConvertStringToBinary to get a buffer of the bytes (based on a string) that you want to write to your file.

   ```
   C#

   var buffer =
   Windows.Security.Cryptography.CryptographicBuffer.ConvertStringToBinary
   (
       "What fools these mortals be",
   Windows.Security.Cryptography.BinaryStringEncoding.Utf8);
   ```

2. Then write the bytes from your buffer to your file by calling the FileIO.WriteBufferAsync method.

   ```
   C#

   await Windows.Storage.FileIO.WriteBufferAsync(sampleFile, buffer);
   ```

Writing text to a file by using a stream (4 steps)

1. First, open the file by calling the StorageFile.OpenAsync method. It returns a stream of the file's content when the open operation completes.

```
C#

var stream = await
sampleFile.OpenAsync(Windows.Storage.FileAccessMode.ReadWrite);
```

2. Next, get an output stream by calling the IRandomAccessStream.GetOutputStreamAt method from the `stream`. If you're using C#, then enclose this in a **using** statement to manage the output stream's lifetime. If you're using C++/WinRT, then you can control its lifetime by enclosing it in a block, or setting it to `nullptr` when you're done with it.

```
C#

using (var outputStream = stream.GetOutputStreamAt(0))
{
    // We'll add more code here in the next step.
}
stream.Dispose(); // Or use the stream variable (see previous code
snippet) with a using statement as well.
```

3. Now add this code (if you're using C#, within the existing **using** statement) to write to the output stream by creating a new DataWriter object and calling the DataWriter.WriteString method.

```
C#

using (var dataWriter = new
Windows.Storage.Streams.DataWriter(outputStream))
{
    dataWriter.WriteString("DataWriter has methods to write to various
types, such as DataTimeOffset.");
}
```

4. Lastly, add this code (if you're using C#, within the inner **using** statement) to save the text to your file with DataWriter.StoreAsync and close the stream with IOutputStream.FlushAsync.

```
C#

await dataWriter.StoreAsync();
await outputStream.FlushAsync();
```

Best practices for writing to a file

For additional details and best practice guidance, see Best practices for writing to files.

Reading from a file

Here's how to read from a file on disk using the **StorageFile** class. The common first step for each of the ways of reading from a file is to get the file with **StorageFolder.GetFileAsync**.

```
C#

Windows.Storage.StorageFolder storageFolder =
    Windows.Storage.ApplicationData.Current.LocalFolder;
Windows.Storage.StorageFile sampleFile =
    await storageFolder.GetFileAsync("sample.txt");
```

Reading text from a file

Read text from your file by calling the **FileIO.ReadTextAsync** method.

```
C#

string text = await Windows.Storage.FileIO.ReadTextAsync(sampleFile);
```

Reading text from a file by using a buffer (2 steps)

1. First, call the **FileIO.ReadBufferAsync** method.

   ```
   C#

   var buffer = await Windows.Storage.FileIO.ReadBufferAsync(sampleFile);
   ```

2. Then use a **DataReader** object to read first the length of the buffer and then its contents.

   ```
   C#

   using (var dataReader =
   Windows.Storage.Streams.DataReader.FromBuffer(buffer))
   {
       string text = dataReader.ReadString(buffer.Length);
   }
   ```

Reading text from a file by using a stream (4 steps)

1. Open a stream for your file by calling the StorageFile.OpenAsync method. It returns a stream of the file's content when the operation completes.

```
C#

var stream = await
sampleFile.OpenAsync(Windows.Storage.FileAccessMode.Read);
```

2. Get the size of the stream to use later.

```
C#

ulong size = stream.Size;
```

3. Get an input stream by calling the IRandomAccessStream.GetInputStreamAt method. Put this in a **using** statement to manage the stream's lifetime. Specify 0 when you call **GetInputStreamAt** to set the position to the beginning of the stream.

```
C#

using (var inputStream = stream.GetInputStreamAt(0))
{
    // We'll add more code here in the next step.
}
```

4. Lastly, add this code within the existing **using** statement to get a DataReader object on the stream then read the text by calling DataReader.LoadAsync and DataReader.ReadString.

```
C#

using (var dataReader = new
Windows.Storage.Streams.DataReader(inputStream))
{
    uint numBytesLoaded = await dataReader.LoadAsync((uint)size);
    string text = dataReader.ReadString(numBytesLoaded);
}
```

See also

- Best practices for writing to files

Best practices for writing to files

Article • 10/20/2022

Important APIs

- FileIO class
- PathIO class

Developers sometimes run into a set of common problems when using the **Write** methods of the **FileIO** and **PathIO** classes to perform file system I/O operations. For example, common problems include:

- A file is partially written.
- The app receives an exception when calling one of the methods.
- The operations leave behind .TMP files with a file name similar to the target file name.

The **Write** methods of the **FileIO** and **PathIO** classes include the following:

- **WriteBufferAsync**
- **WriteBytesAsync**
- **WriteLinesAsync**
- **WriteTextAsync**

This article provides details about how these methods work so developers understand better when and how to use them. This article provides guidelines and does not attempt to provide a solution for all possible file I/O problems.

> ⓘ **Note**
>
> This article focuses on the **FileIO** methods in examples and discussions. However, the **PathIO** methods follow a similar pattern and most of the guidance in this article applies to those methods too.

Convenience vs. control

A **StorageFile** object is not a file handle like the native Win32 programming model. Instead, a **StorageFile** is a representation of a file with methods to manipulate its contents.

Understanding this concept is useful when performing I/O with a **StorageFile**. For example, the Writing to a file section presents three ways to write to a file:

- Using the **FileIO.WriteTextAsync** method.
- By creating a buffer and then calling the **FileIO.WriteBufferAsync** method.
- The four-step model using a stream:

    1. Open the file to get a stream.
    2. Get an output stream.
    3. Create a **DataWriter** object and call the corresponding **Write** method.
    4. Commit the data in the data writer and flush the output stream.

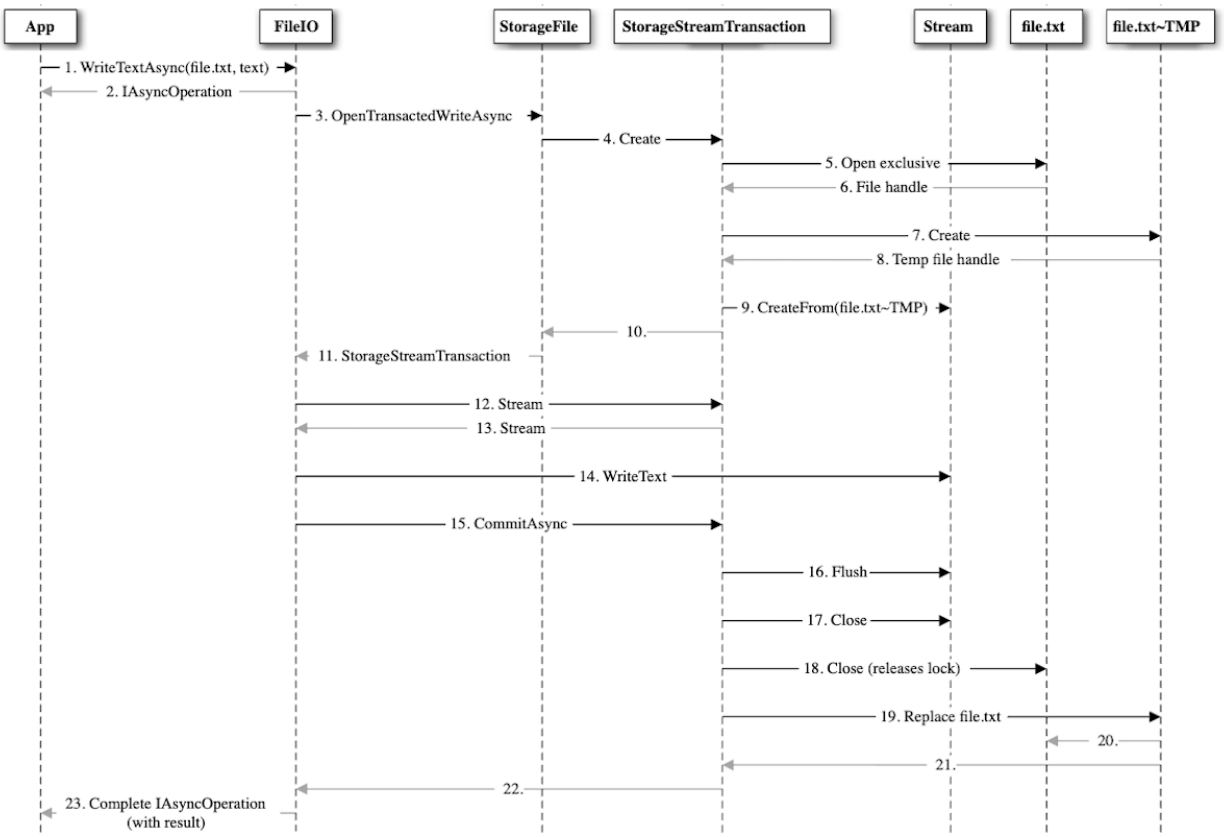
The first two scenarios are the ones most commonly used by apps. Writing to the file in a single operation is easier to code and maintain, and it also removes the responsibility of the app from dealing with many of the complexities of file I/O. However, this convenience comes at a cost: the loss of control over the entire operation and the ability to catch errors at specific points.

The transactional model

The **Write** methods of the **FileIO** and **PathIO** classes wrap the steps on the third write model described above, with an added layer. This layer is encapsulated in a storage transaction.

To protect the integrity of the original file in case something goes wrong while writing the data, the **Write** methods use a transactional model by opening the file using **OpenTransactedWriteAsync**. This process creates a **StorageStreamTransaction** object. After this transaction object is created, the APIs write the data following a similar fashion to the File Access sample or the code example in the **StorageStreamTransaction** article.

The following diagram illustrates the underlying tasks performed by the **WriteTextAsync** method in a successful write operation. This illustration provides a simplified view of the operation. For example, it skips steps such as text encoding and async completion on different threads.



The advantages of using the **Write** methods of the **FileIO** and **PathIO** classes instead of the more complex four-step model using a stream are:

- One API call to handle all the intermediate steps, including errors.
- The original file is kept if something goes wrong.
- The system state will try to be kept as clean as possible.

However, with so many possible intermediate points of failure, there's an increased chance of failure. When an error occurs it may be difficult to understand where the process failed. The following sections present some of the failures you might encounter when using the **Write** methods and provide possible solutions.

Common error codes for Write methods of the FileIO and PathIO classes

This table presents common error codes that app developers encounter when using the **Write** methods. The steps in the table correspond to steps in the previous diagram.

Error name (value)	Steps	Causes	Solutions
ERROR_ACCESS_DENIED (0X80070005)	5	The original file might be marked for deletion, possibly from a previous operation.	Retry the operation. Ensure access to the file is synchronized.

Error name (value)	Steps	Causes	Solutions
ERROR_SHARING_VIOLATION (0x80070020)	5	The original file is opened by another exclusive write.	Retry the operation. Ensure access to the file is synchronized.
ERROR_UNABLE_TO_REMOVE_REPLACED (0x80070497)	19-20	The original file (file.txt) could not be replaced because it is in use. Another process or operation gained access to the file before it could be replaced.	Retry the operation. Ensure access to the file is synchronized.
ERROR_DISK_FULL (0x80070070)	7, 14, 16, 20	The transacted model creates an extra file, and this consumes extra storage.	
ERROR_OUTOFMEMORY (0x8007000E)	14, 16	This can happen due to multiple outstanding I/O operations or large file sizes.	A more granular approach by controlling the stream might resolve the error.
E_FAIL (0x80004005)	Any	Miscellaneous	Retry the operation. If it still fails, it might be a platform error and the app should terminate because it's in an inconsistent state.

Other considerations for file states that might lead to errors

Apart from errors returned by the **Write** methods, here are some guidelines on what an app can expect when writing to a file.

Data was written to the file if and only if operation completed

Your app should not make any assumption about data in the file while a write operation is in progress. Trying to access the file before an operation completes might lead to

inconsistent data. Your app should be responsible of tracking outstanding I/Os.

Readers

If the file that being written to is also being used by a polite reader (that is, opened with FileAccessMode.Read, subsequent reads will fail with an error ERROR_OPLOCK_HANDLE_CLOSED (0x80070323). Sometimes apps retry opening the file for read again while the **Write** operation is ongoing. This might result in a race condition on which the **Write** ultimately fails when trying to overwrite the original file because it cannot be replaced.

Files from KnownFolders

Your app might not be the only app that is trying to access a file that resides on any of the KnownFolders. There's no guarantee that if the operation is successful, the contents an app wrote to the file will remain constant the next time it tries to read the file. Also, sharing or access denied errors become more common under this scenario.

Conflicting I/O

The chances of concurrency errors can be lowered if our app uses the **Write** methods for files in its local data, but some caution is still required. If multiple **Write** operations are being sent concurrently to the file, there's no guarantee about what data ends up in the file. To mitigate this, we recommend that your app serializes **Write** operations to the file.

~TMP files

Occasionally, if the operation is forcefully cancelled (for example, if the app was suspended or terminated by the OS), the transaction is not committed or closed appropriately. This can leave behind files with a (.~TMP) extension. Consider deleting these temporary files (if they exist in the app's local data) when handling the app activation.

Considerations based on file types

Some errors can become more prevalent depending on the type of files, the frequency on which they're accessed, and their file size. Generally, there are three categories of files your app can access:

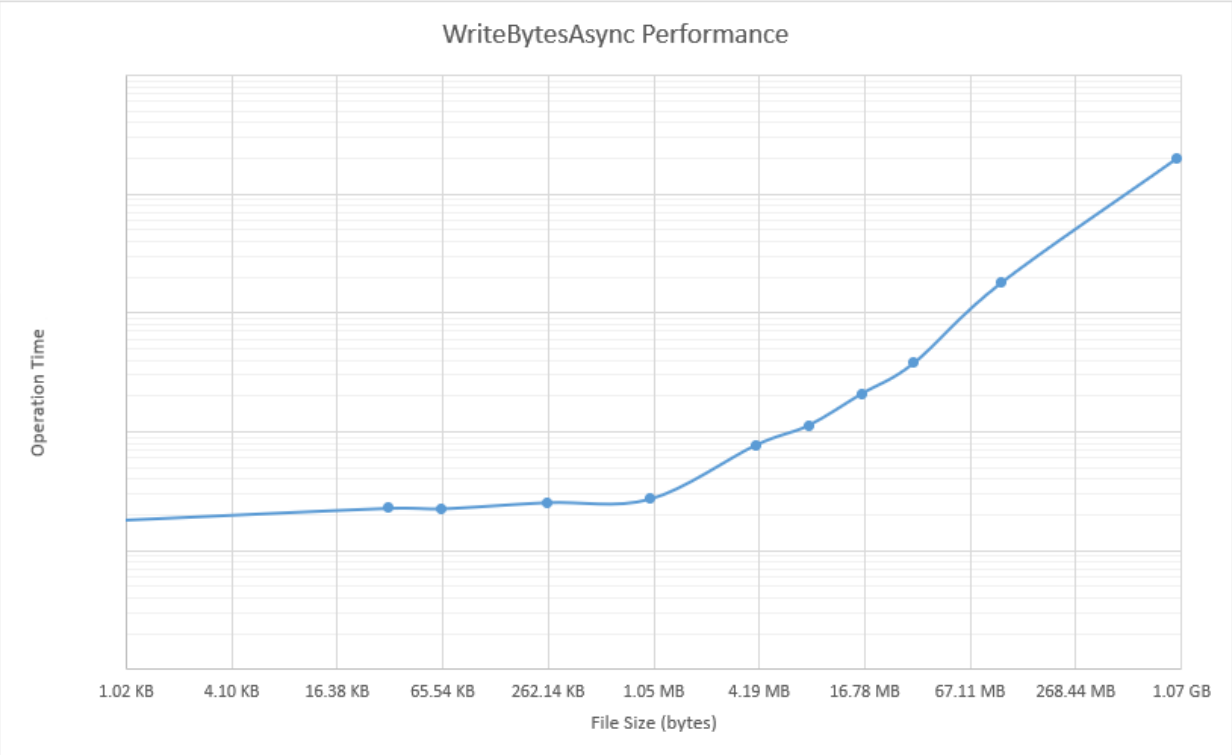
- Files created and edited by the user in your app's local data folder. These are created and edited only while using your app, and they exist only within the app.
- App metadata. Your app uses these files to keep track of its own state.
- Other files in locations of the file system where your app has declared capabilities to access. These are most commonly located in one of the **KnownFolders**.

Your app has full control on the first two categories of files, because they're part of your app's package files and are accessed by your app exclusively. For files in the last category, your app must be aware that other apps and OS services may be accessing the files concurrently.

Depending on the app, access to the files can vary on frequency:

- Very low. These are usually files that are opened once when the app launches and are saved when the app is suspended.
- Low. These are files that the user is specifically taking an action on (such as save or load).
- Medium or high. These are files in which the app must constantly update data (for example, autosave features or constant metadata tracking).

For file size, consider the performance data in the following chart for the **WriteBytesAsync** method. This chart compares the time to complete an operation vs file size, over an average performance of 10000 operations per file size in a controlled environment.



The time values on the y-axis are omitted intentionally from this chart because different hardware and configurations will yield different absolute time values. However, we have consistently observed these trends in our tests:

- For very small files (<= 1 MB): The time to complete the operations is consistently fast.
- For larger files (> 1 MB): The time to complete the operations starts to increase exponentially.

I/O during app suspension

Your app must designed to handle suspension if you want to keep state information or metadata for use in later sessions. For background information about app suspension, see App lifecycle and this blog post.

Unless the OS grants extended execution to your app, when your app is suspended it has 5 seconds to release all its resources and save its data. For the best reliability and user experience, always assume the time you have to handle suspension tasks is limited. Keep in mind the following guidelines during the 5 second time period for handling suspension tasks:

- Try to keep I/O to a minimum to avoid race conditions caused by flushing and release operations.
- Avoid writing files that require hundreds of milliseconds or more to write.
- If your app uses the **Write** methods, keep in mind all the intermediate steps that these methods require.

If your app operates on a small amount of state data during suspension, in most cases you can use the **Write** methods to flush the data. However, if your app uses a large amount of state data, consider using streams to directly store your data. This can help reduce the delay introduced by the transactional model of the **Write** methods.

For an example, see the BasicSuspension sample.

Other examples and resources

Here are several examples and other resources for specific scenarios.

Code example for retrying file I/O example

The following is a pseudo-code example on how to retry a write (C#), assuming the write is to be done after the user picks a file for saving:

C#

```
Windows.Storage.Pickers.FileSavePicker savePicker = new
Windows.Storage.Pickers.FileSavePicker();
savePicker.FileTypeChoices.Add("Plain Text", new List<string>() { ".txt" });
Windows.Storage.StorageFile file = await savePicker.PickSaveFileAsync();

Int32 retryAttempts = 5;

const Int32 ERROR_ACCESS_DENIED = unchecked((Int32)0x80070005);
const Int32 ERROR_SHARING_VIOLATION = unchecked((Int32)0x80070020);

if (file != null)
{
    // Application now has read/write access to the picked file.
    while (retryAttempts > 0)
    {
        try
        {
            retryAttempts--;
            await Windows.Storage.FileIO.WriteTextAsync(file, "Text to write
to file");
            break;
        }
        catch (Exception ex) when ((ex.HResult == ERROR_ACCESS_DENIED) ||
                                   (ex.HResult == ERROR_SHARING_VIOLATION))
        {
            // This might be recovered by retrying, otherwise let the
exception be raised.
            // The app can decide to wait before retrying.
        }
    }
}
else
{
    // The operation was cancelled in the picker dialog.
}
```

Synchronize access to the file

The Parallel Programming with .NET blog is a great resource for guidance about parallel programming. In particular, the post about AsyncReaderWriterLock describes how to maintain exclusive access to a file for writes while allowing concurrent read access. Keep in mind that serializing I/O will impact performance.

See also

- Create, write, and read a file

Get file properties

Article • 10/20/2022

Important APIs

- StorageFile.GetBasicPropertiesAsync
- StorageFile.Properties
- StorageItemContentProperties.RetrievePropertiesAsync

Get properties—top-level, basic, and extended—for a file represented by a StorageFile object.

> **Note**
>
> For a complete sample, see the **File access sample** .

Prerequisites

- **Understand async programming for Universal Windows Platform (UWP) apps**

  You can learn how to write asynchronous apps in C# or Visual Basic, see Call asynchronous APIs in C# or Visual Basic. To learn how to write asynchronous apps in C++, see Asynchronous programming in C++.

- **Access permissions to the location**

  For example, the code in these examples require the **picturesLibrary** capability, but your location may require a different capability or no capability at all. To learn more, see File access permissions.

Getting a file's top-level properties

Many top-level file properties are accessible as members of the StorageFile class. These properties include the files attributes, content type, creation date, display name, file type, and so on.

> **Note**
>
> Remember to declare the **picturesLibrary** capability.

This example enumerates all of the files in the Pictures library, accessing a few of each file's top-level properties.

C#

```
// Enumerate all files in the Pictures library.
var folder = Windows.Storage.KnownFolders.PicturesLibrary;
var query = folder.CreateFileQuery();
var files = await query.GetFilesAsync();

foreach (Windows.Storage.StorageFile file in files)
{
    StringBuilder fileProperties = new StringBuilder();

    // Get top-level file properties.
    fileProperties.AppendLine("File name: " + file.Name);
    fileProperties.AppendLine("File type: " + file.FileType);
}
```

Getting a file's basic properties

Many basic file properties are obtained by first calling the StorageFile.GetBasicPropertiesAsync method. This method returns a BasicProperties object, which defines properties for the size of the item (file or folder) as well as when the item was last modified.

This example enumerates all of the files in the Pictures library, accessing a few of each file's basic properties.

C#

```
// Enumerate all files in the Pictures library.
var folder = Windows.Storage.KnownFolders.PicturesLibrary;
var query = folder.CreateFileQuery();
var files = await query.GetFilesAsync();

foreach (Windows.Storage.StorageFile file in files)
{
    StringBuilder fileProperties = new StringBuilder();

    // Get file's basic properties.
    Windows.Storage.FileProperties.BasicProperties basicProperties =
        await file.GetBasicPropertiesAsync();
    string fileSize = string.Format("{0:n0}", basicProperties.Size);
    fileProperties.AppendLine("File size: " + fileSize + " bytes");
    fileProperties.AppendLine("Date modified: " +
basicProperties.DateModified);
}
```

Getting a file's extended properties

Aside from the top-level and basic file properties, there are many properties associated with the file's contents. These extended properties are accessed by calling the **BasicProperties.RetrievePropertiesAsync** method. (A **BasicProperties** object is obtained by calling the **StorageFile.Properties** property.) While top-level and basic file properties are accessible as properties of a class—**StorageFile** and **BasicProperties**, respectively—extended properties are obtained by passing an IEnumerable collection of String objects representing the names of the properties that are to be retrieved to the **BasicProperties.RetrievePropertiesAsync** method. This method then returns an IDictionary collection. Each extended property is then retrieved from the collection by name or by index.

This example enumerates all of the files in the Pictures library, specifies the names of desired properties (**DataAccessed** and **FileOwner**) in a List object, passes that List object to **BasicProperties.RetrievePropertiesAsync** to retrieve those properties, and then retrieves those properties by name from the returned IDictionary object.

See the Windows Core Properties for a complete list of a file's extended properties.

C#

```
const string dateAccessedProperty = "System.DateAccessed";
const string fileOwnerProperty = "System.FileOwner";

// Enumerate all files in the Pictures library.
var folder = KnownFolders.PicturesLibrary;
var query = folder.CreateFileQuery();
var files = await query.GetFilesAsync();

foreach (Windows.Storage.StorageFile file in files)
{
    StringBuilder fileProperties = new StringBuilder();

    // Define property names to be retrieved.
    var propertyNames = new List<string>();
    propertyNames.Add(dateAccessedProperty);
    propertyNames.Add(fileOwnerProperty);

    // Get extended properties.
    IDictionary<string, object> extraProperties =
        await file.Properties.RetrievePropertiesAsync(propertyNames);

    // Get date-accessed property.
    var propValue = extraProperties[dateAccessedProperty];
    if (propValue != null)
    {
        fileProperties.AppendLine("Date accessed: " + propValue);
    }
```

```
    // Get file-owner property.
    propValue = extraProperties[fileOwnerProperty];
    if (propValue != null)
    {
        fileProperties.AppendLine("File owner: " + propValue);
    }
}
```

Open files and folders with a picker

Article • 02/08/2024

Important APIs

- FileOpenPicker
- FolderPicker
- StorageFile

Access files and folders by letting the user interact with a picker. You can use the FileOpenPicker and FileSavePicker classes to access files, and the FolderPicker to access a folder.

> **Note**
>
> For a complete sample, see the **File picker sample**.

> **Note**
>
> In a desktop app (which includes WinUI 3 apps), you can use file and folder pickers from **Windows.Storage.Pickers**. However, if the desktop app requires elevation to run, you'll need a different approach because these APIs aren't designed to be used in an elevated app. For an example, see **FileSavePicker**.

Prerequisites

- **Understand async programming for Universal Windows Platform (UWP) apps**

  You can learn how to write asynchronous apps in C# or Visual Basic, see Call asynchronous APIs in C# or Visual Basic. To learn how to write asynchronous apps in C++, see Asynchronous programming in C++.

- **Access permissions to the location**

  See File access permissions.

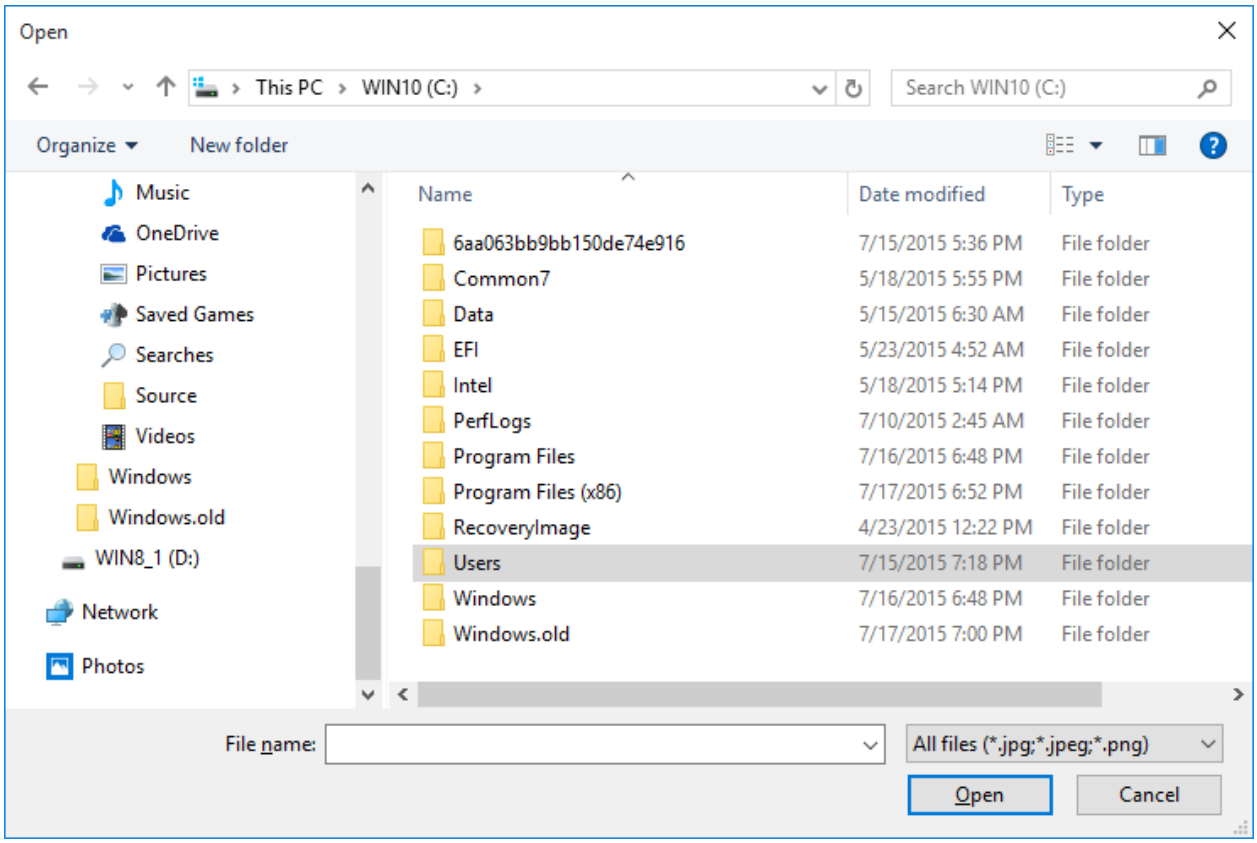
File picker UI

A file picker displays information to orient users and provide a consistent experience when opening or saving files.

That information includes:

- The current location
- The item or items that the user picked
- A tree of locations that the user can browse to. These locations include file system locations—such as the Music or Downloads folder—as well as apps that implement the file picker contract (such as Camera, Photos, and Microsoft OneDrive).

An email app might display a file picker for the user to pick attachments.



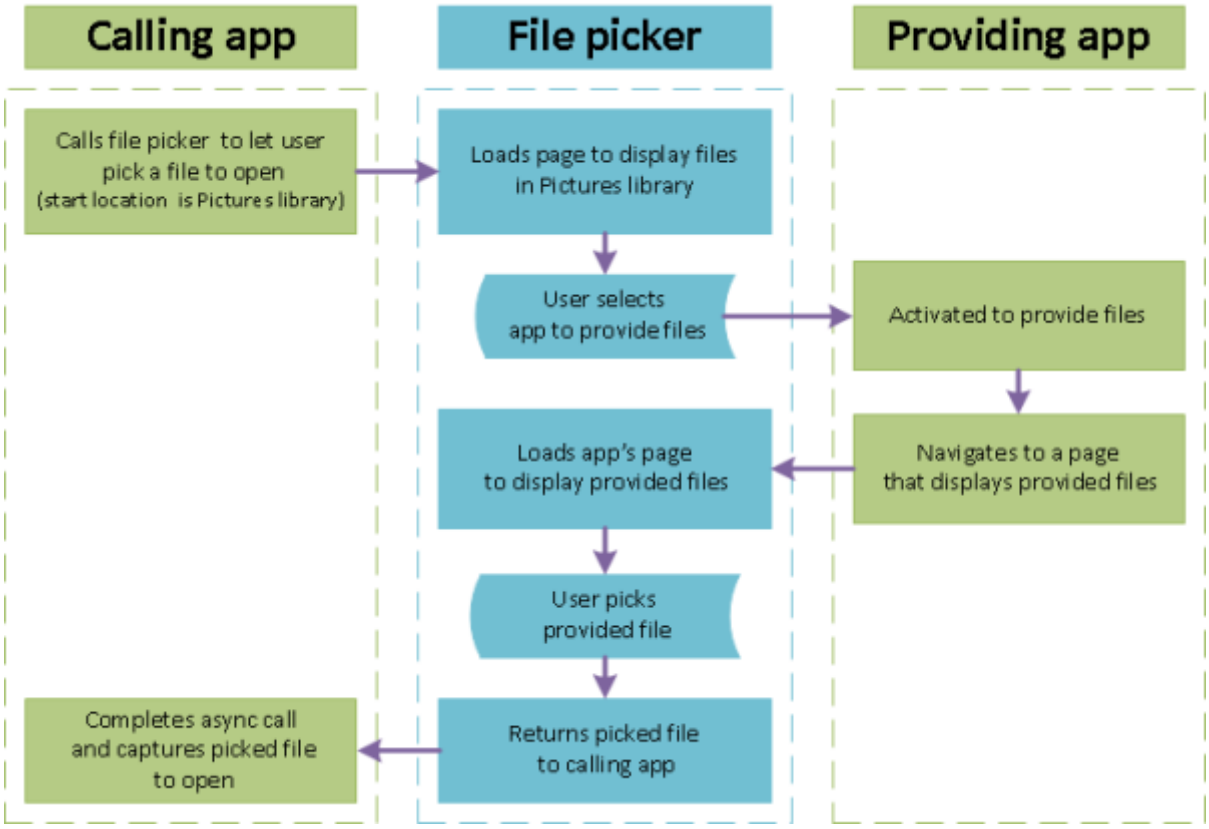
How pickers work

With a picker your app can access, browse, and save files and folders on the user's system. Your app receives those picks as **StorageFile** and **StorageFolder** objects, which you can then operate on.

The picker uses a single, unified interface to let the user pick files and folders from the file system or from other apps. Files picked from other apps are like files from the file system: they are returned as **StorageFile** objects. In general, your app can operate on them in the same ways as other objects. Other apps make files available by participating in file picker contracts. If you want your app to provide files, a save location, or file updates to other apps, see Integrating with file picker contracts.

For example, you might call the file picker in your app so that your user can open a file. This makes your app the calling app. The file picker interacts with the system and/or

other apps to let the user navigate and pick the file. When your user chooses a file, the file picker returns that file to your app. Here's the process for the case of the user choosing a file from a providing app, such as OneDrive.



Pick a single file: complete code listing

cs

```
var picker = new Windows.Storage.Pickers.FileOpenPicker();
picker.ViewMode = Windows.Storage.Pickers.PickerViewMode.Thumbnail;
picker.SuggestedStartLocation =
Windows.Storage.Pickers.PickerLocationId.PicturesLibrary;
picker.FileTypeFilter.Add(".jpg");
picker.FileTypeFilter.Add(".jpeg");
picker.FileTypeFilter.Add(".png");

Windows.Storage.StorageFile file = await picker.PickSingleFileAsync();
if (file != null)
{
    // Application now has read/write access to the picked file
    this.textBlock.Text = "Picked photo: " + file.Name;
}
else
{
    this.textBlock.Text = "Operation cancelled.";
}
```

Pick a single file: step-by-step

Using a file picker involves creating and customizing a file picker object, and then showing the file picker so the user can pick one or more items.

1. **Create and customize a FileOpenPicker**

```
cs
```

```cs
var picker = new Windows.Storage.Pickers.FileOpenPicker();
picker.ViewMode = Windows.Storage.Pickers.PickerViewMode.Thumbnail;
picker.SuggestedStartLocation =
Windows.Storage.Pickers.PickerLocationId.PicturesLibrary;
picker.FileTypeFilter.Add(".jpg");
picker.FileTypeFilter.Add(".jpeg");
picker.FileTypeFilter.Add(".png");
```

   Set properties on the file picker object relevant to your users and app.

   This example creates a rich, visual display of pictures in a convenient location that the user can pick from by setting three properties: ViewMode, SuggestedStartLocation, and FileTypeFilter.

   - Setting ViewMode to the PickerViewMode **Thumbnail** enum value creates a rich, visual display by using picture thumbnails to represent files in the file picker. Do this for picking visual files such as pictures or videos. Otherwise, use PickerViewMode.List. A hypothetical email app with **Attach Picture or Video** and **Attach Document** features would set the **ViewMode** appropriate to the feature before showing the file picker.

   - Setting SuggestedStartLocation to Pictures using PickerLocationId.PicturesLibrary starts the user in a location where they're likely to find pictures. Set **SuggestedStartLocation** to a location appropriate for the type of file being picked, for example Music, Pictures, Videos, or Documents. From the start location, the user can navigate to other locations.

   - Using FileTypeFilter to specify file types keeps the user focused on picking files that are relevant. To replace previous file types in the **FileTypeFilter** with new entries, use ReplaceAll instead of Add.

2. **Show the FileOpenPicker**

   - **To pick a single file**

```
cs
```

```
Windows.Storage.StorageFile file = await
picker.PickSingleFileAsync();
if (file != null)
{
    // Application now has read/write access to the picked file
    this.textBlock.Text = "Picked photo: " + file.Name;
}
else
{
    this.textBlock.Text = "Operation cancelled.";
}
```

- To pick multiple files

cs

```
var files = await picker.PickMultipleFilesAsync();
if (files.Count > 0)
{
    StringBuilder output = new StringBuilder("Picked files:\n");

    // Application now has read/write access to the picked file(s)
    foreach (Windows.Storage.StorageFile file in files)
    {
        output.Append(file.Name + "\n");
    }
    this.textBlock.Text = output.ToString();
}
else
{
    this.textBlock.Text = "Operation cancelled.";
}
```

Pick a folder: complete code listing

cs

```
var folderPicker = new Windows.Storage.Pickers.FolderPicker();
folderPicker.SuggestedStartLocation =
Windows.Storage.Pickers.PickerLocationId.Desktop;
folderPicker.FileTypeFilter.Add("*");

Windows.Storage.StorageFolder folder = await
folderPicker.PickSingleFolderAsync();
if (folder != null)
{
    // Application now has read/write access to all contents in the picked
folder
    // (including other sub-folder contents)
```

```
    Windows.Storage.AccessCache.StorageApplicationPermissions.
    FutureAccessList.AddOrReplace("PickedFolderToken", folder);
    this.textBlock.Text = "Picked folder: " + folder.Name;
}
else
{
    this.textBlock.Text = "Operation cancelled.";
}
```

> **Tip**
>
> Whenever your app accesses a file or folder through a picker, add it to your app's **FutureAccessList** or **MostRecentlyUsedList** to keep track of it. You can learn more about using these lists in **How to track recently-used files and folders**.

See also

Windows.Storage.Pickers

Files, folders, and libraries

Integrating with file picker contracts

Save a file with a picker

Article • 02/08/2024

Important APIs

- FileSavePicker
- StorageFile

Use FileSavePicker to let users specify the name and location where they want your app to save a file.

> **Note**
>
> For a complete sample, see the **File picker sample**.

> **Note**
>
> In a desktop app (which includes WinUI 3 apps), you can use file and folder pickers from **Windows.Storage.Pickers**. However, if the desktop app requires elevation to run, you'll need a different approach because these APIs aren't designed to be used in an elevated app. For an example, see **FileSavePicker**.

Prerequisites

- **Understand async programming for Universal Windows Platform (UWP) apps**

  You can learn how to write asynchronous apps in C# or Visual Basic, see Call asynchronous APIs in C# or Visual Basic. To learn how to write asynchronous apps in C++, see Asynchronous programming in C++.

- **Access permissions to the location**

  See File access permissions.

FileSavePicker: step-by-step

Use a FileSavePicker so that your users can specify the name, type, and location of a file to save. Create, customize, and show a file picker object, and then save data via the returned StorageFile object that represents the file picked.

1. **Create and customize the FileSavePicker**

```cs
var savePicker = new Windows.Storage.Pickers.FileSavePicker();
savePicker.SuggestedStartLocation =
    Windows.Storage.Pickers.PickerLocationId.DocumentsLibrary;
// Dropdown of file types the user can save the file as
savePicker.FileTypeChoices.Add("Plain Text", new List<string>() {
".txt" });
// Default file name if the user does not type one in or select a file
to replace
savePicker.SuggestedFileName = "New Document";
```

Set properties on the file picker object that are relevant to your users and your app. This example sets three properties: **SuggestedStartLocation**, **FileTypeChoices** and **SuggestedFileName**.

- Because our user is saving a document or text file, the sample sets **SuggestedStartLocation** to the app's local folder by using **LocalFolder**. Set **SuggestedStartLocation** to a location appropriate for the type of file being saved, for example Music, Pictures, Videos, or Documents. From the start location, the user can navigate to other locations.

- Because we want to make sure our app can open the file after it is saved, we use **FileTypeChoices** to specify file types that the sample supports (Microsoft Word documents and text files). Make sure all the file types that you specify are supported by your app. Users will be able to save their file as any of the file types you specify. They can also change the file type by selecting another of the file types that you specified. The first file type choice in the list will be selected by default: to control that, set the **DefaultFileExtension** property.

> **Note**
>
> The file picker also uses the currently selected file type to filter which files it displays, so that only file types that match the selected files types are displayed to the user.

- To save the user some typing, the example sets a **SuggestedFileName**. Make your suggested file name relevant to the file being saved. For example, like Word, you can suggest the existing file name if there is one, or the first line of a document if the user is saving a file that does not yet have a name.

> **Note**
>
> **FileSavePicker** objects display the file picker using the **PickerViewMode.List** view mode.

2. **Show the FileSavePicker and save to the picked file**

   Display the file picker by calling **PickSaveFileAsync**. After the user specifies the name, file type, and location, and confirms to save the file, **PickSaveFileAsync** returns a **StorageFile** object that represents the saved file. You can capture and process this file now that you have read and write access to it.

```cs
Windows.Storage.StorageFile file = await
savePicker.PickSaveFileAsync();
if (file != null)
{
    // Prevent updates to the remote version of the file until
    // we finish making changes and call CompleteUpdatesAsync.
    Windows.Storage.CachedFileManager.DeferUpdates(file);
    // write to file
    await Windows.Storage.FileIO.WriteTextAsync(file, "file contents");
    // Let Windows know that we're finished changing the file so
    // the other app can update the remote version of the file.
    // Completing updates may require Windows to ask for user input.
    Windows.Storage.Provider.FileUpdateStatus status =
        await
Windows.Storage.CachedFileManager.CompleteUpdatesAsync(file);
    if (status == Windows.Storage.Provider.FileUpdateStatus.Complete)
    {
        this.textBlock.Text = "File " + file.Name + " was saved.";
    }
    else
    {
        this.textBlock.Text = "File " + file.Name + " couldn't be
saved.";
    }
}
else
{
    this.textBlock.Text = "Operation cancelled.";
}
```

The example checks that the file is valid and writes its own file name into it. Also see Creating, writing, and reading a file.

> **Tip**

You should always check the saved file to make sure it is valid before you perform any other processing. Then, you can save content to the file as appropriate for your app, and provide appropriate behavior if the picked file is not valid.

See also

Windows.Storage.Pickers

Files, folders, and libraries

Integrating with file picker contracts

Creating, writing, and reading a file

Accessing HomeGroup content

Article • 10/20/2022

Important APIs

- Windows.Storage.KnownFolders class

Access content stored in the user's HomeGroup folder, including pictures, music, and videos.

Prerequisites

- **Understand async programming for Universal Windows Platform (UWP) apps**

  You can learn how to write asynchronous apps in C# or Visual Basic, see Call asynchronous APIs in C# or Visual Basic. To learn how to write asynchronous apps in C++, see Asynchronous programming in C++.

- **App capability declarations**

  To access HomeGroup content, the user's machine must have a HomeGroup set up and your app must have at least one of the following capabilities: **picturesLibrary**, **musicLibrary**, or **videosLibrary**. When your app accesses the HomeGroup folder, it will see only the libraries that correspond to the capabilities declared in your app's manifest. To learn more, see File access permissions.

  > **Note**
  >
  > Content in the Documents library of a HomeGroup isn't visible to your app regardless of the capabilities declared in your app's manifest and regardless of the user's sharing settings.

- **Understand how to use file pickers**

  You typically use the file picker to access files and folders in the HomeGroup. To learn how to use the file picker, see Open files and folders with a picker.

- **Understand file and folder queries**

  You can use queries to enumerate files and folders in the HomeGroup. To learn about file and folder queries, see Enumerating and querying files and folders.

Open the file picker at the HomeGroup

Follow these steps to open an instance of the file picker that lets the user pick files and folders from the HomeGroup:

1. **Create and customize the file picker**

   Use **FileOpenPicker** to create the file picker, and then set the picker's **SuggestedStartLocation** to **PickerLocationId.HomeGroup**. Or, set other properties that are relevant to your users and your app. For guidelines to help you decide how to customize the file picker, see Guidelines and checklist for file pickers

   This example creates a file picker that opens at the HomeGroup, includes files of any type, and displays the files as thumbnail images:

   ```
   cs

   Windows.Storage.Pickers.FileOpenPicker picker = new
   Windows.Storage.Pickers.FileOpenPicker();
   picker.ViewMode = Windows.Storage.Pickers.PickerViewMode.Thumbnail;
   picker.SuggestedStartLocation =
   Windows.Storage.Pickers.PickerLocationId.HomeGroup;
   picker.FileTypeFilter.Clear();
   picker.FileTypeFilter.Add("*");
   ```

2. **Show the file picker and process the picked file.**

   After you create and customize the file picker, let the user pick one file by calling **FileOpenPicker.PickSingleFileAsync**, or multiple files by calling **FileOpenPicker.PickMultipleFilesAsync**.

   This example displays the file picker to let the user pick one file:

   ```
   cs

   Windows.Storage.StorageFile file = await picker.PickSingleFileAsync();

   if (file != null)
   {
       // Do something with the file.
   }
   else
   {
       // No file returned. Handle the error.
   }
   ```

Search the HomeGroup for files

This section shows how to find HomeGroup items that match a query term provided by the user.

1. **Get the query term from the user.**

   Here we get a query term that the user has entered into a **TextBox** control called `searchQueryTextBox`:

   ```cs
   string queryTerm = this.searchQueryTextBox.Text;
   ```

2. **Set the query options and search filter.**

   Query options determine how the search results are sorted, while the search filter determines which items are included in the search results.

   This example sets query options that sort the search results by relevance and then the date modified. The search filter is the query term that the user entered in the previous step:

   ```cs
   Windows.Storage.Search.QueryOptions queryOptions =
           new Windows.Storage.Search.QueryOptions
               (Windows.Storage.Search.CommonFileQuery.OrderBySearchRank,
   null);
   queryOptions.UserSearchFilter = queryTerm.Text;
   Windows.Storage.Search.StorageFileQueryResult queryResults =

   Windows.Storage.KnownFolders.HomeGroup.CreateFileQueryWithOptions(query
   Options);
   ```

3. **Run the query and process the results.**

   The following example runs the search query in the HomeGroup and saves the names of any matching files as a list of strings.

   ```cs
   System.Collections.Generic.IReadOnlyList<Windows.Storage.StorageFile>
   files =
       await queryResults.GetFilesAsync();

   if (files.Count > 0)
   ```

```
{
    outputString += (files.Count == 1) ? "One file found\n" :
files.Count.ToString() + " files found\n";
    foreach (Windows.Storage.StorageFile file in files)
    {
        outputString += file.Name + "\n";
    }
}
```

Search the HomeGroup for a particular user's shared files

This section shows you how to find HomeGroup files that are shared by a particular user.

1. **Get a collection of HomeGroup users.**

   Each of the first-level folders in the HomeGroup represents an individual HomeGroup user. So, to get the collection of HomeGroup users, call GetFoldersAsync retrieve the top-level HomeGroup folders.

   ```
   cs

   System.Collections.Generic.IReadOnlyList<Windows.Storage.StorageFolder>
   hgFolders =
       await Windows.Storage.KnownFolders.HomeGroup.GetFoldersAsync();
   ```

2. **Find the target user's folder, and then create a file query scoped to that user's folder.**

   The following example iterates through the retrieved folders to find the target user's folder. Then, it sets query options to find all files in the folder, sorted first by relevance and then by the date modified. The example builds a string that reports the number of files found, along with the names of the files.

   ```
   cs

   bool userFound = false;
   foreach (Windows.Storage.StorageFolder folder in hgFolders)
   {
       if (folder.DisplayName == targetUserName)
       {
           // Found the target user's folder, now find all files in the
   folder.
           userFound = true;
           Windows.Storage.Search.QueryOptions queryOptions =
               new Windows.Storage.Search.QueryOptions
   ```

```
(Windows.Storage.Search.CommonFileQuery.OrderBySearchRank, null);
        queryOptions.UserSearchFilter = "*";
        Windows.Storage.Search.StorageFileQueryResult queryResults =
            folder.CreateFileQueryWithOptions(queryOptions);

System.Collections.Generic.IReadOnlyList<Windows.Storage.StorageFile>
files =
            await queryResults.GetFilesAsync();

        if (files.Count > 0)
        {
            string outputString = "Searched for files belonging to " +
targetUserName + "'\n";
            outputString += (files.Count == 1) ? "One file found\n" :
files.Count.ToString() + " files found\n";
            foreach (Windows.Storage.StorageFile file in files)
            {
                outputString += file.Name + "\n";
            }
        }
    }
}
```

Stream video from the HomeGroup

Follow these steps to stream video content from the HomeGroup:

1. **Include a MediaElement in your app.**

   A **MediaElement** lets you play back audio and video content in your app. For more information on audio and video playback, see Create custom transport controls and Audio, video, and camera.

   HTML

   ```
   <Grid x:Name="Output" HorizontalAlignment="Left"
   VerticalAlignment="Top" Grid.Row="1">
       <MediaElement x:Name="VideoBox" HorizontalAlignment="Left"
   VerticalAlignment="Top" Margin="0" Width="400" Height="300"/>
   </Grid>
   ```

2. **Open a file picker at the HomeGroup and apply a filter that includes video files in the formats that your app supports.**

   This example includes .mp4 and .wmv files in the file open picker.

   cs

```
Windows.Storage.Pickers.FileOpenPicker picker = new
Windows.Storage.Pickers.FileOpenPicker();
picker.ViewMode = Windows.Storage.Pickers.PickerViewMode.Thumbnail;
picker.SuggestedStartLocation =
Windows.Storage.Pickers.PickerLocationId.HomeGroup;
picker.FileTypeFilter.Clear();
picker.FileTypeFilter.Add(".mp4");
picker.FileTypeFilter.Add(".wmv");
Windows.Storage.StorageFile file = await picker.PickSingleFileAsync();
```

3. **Open the user's file selection for read access, and set the file stream as the source for the MediaElement**, and then play the file.

```cs
if (file != null)
{
    var stream = await
file.OpenAsync(Windows.Storage.FileAccessMode.Read);
    VideoBox.SetSource(stream, file.ContentType);
    VideoBox.Stop();
    VideoBox.Play();
}
else
{
    // No file selected. Handle the error here.
}
```

Feedback

Determining availability of Microsoft OneDrive files

Article • 10/20/2022

Important APIs

- FileIO class
- StorageFile class
- StorageFile.IsAvailable property

Determine if a Microsoft OneDrive file is available using the StorageFile.IsAvailable property.

Prerequisites

- **Understand async programming for Universal Windows Platform (UWP) apps**

  You can learn how to write asynchronous apps in C# or Visual Basic, see Call asynchronous APIs in C# or Visual Basic. To learn how to write asynchronous apps in C++, see Asynchronous programming in C++.

- **App capability declarations**

  See File access permissions.

Using the StorageFile.IsAvailable property

Users are able to mark OneDrive files as either available-offline (default) or online-only. This capability enables users to move large files (such as pictures and videos) to their OneDrive, mark them as online-only, and save disk space (the only thing kept locally is a metadata file).

StorageFile.IsAvailable, is used to determine if a file is currently available. The following table shows the value of the **StorageFile.IsAvailable** property in various scenarios.

Expand table

Type of file	Online	Metered network	Offline
Local file	True	True	True

Type of file	Online	Metered network	Offline
OneDrive file marked as available-offline	True	True	True
OneDrive file marked as online-only	True	Based on user settings	False
Network file	True	Based on user settings	False

The following steps illustrate how to determine if a file is currently available.

1. Declare a capability appropriate for the library you want to access.
2. Include the **Windows.Storage** namespace. This namespace includes the types for managing files, folders, and application settings. It also includes the needed **StorageFile** type.
3. Acquire a **StorageFile** object for the desired file(s). If you are enumerating a library, this step is usually accomplished by calling the **StorageFolder.CreateFileQuery** method and then calling the resulting **StorageFileQueryResult** object's **GetFilesAsync** method. The **GetFilesAsync** method returns an IReadOnlyList collection of **StorageFile** objects.
4. Once you have the access to a **StorageFile** object representing the desired file(s), the value of the **StorageFile.IsAvailable** property reflects whether or not the file is available.

The following generic method illustrates how to enumerate any folder and return the collection of **StorageFile** objects for that folder. The calling method then iterates over the returned collection referencing the **StorageFile.IsAvailable** property for each file.

cs

```
/// <summary>
/// Generic function that retrieves all files from the specified folder.
/// </summary>
/// <param name="folder">The folder to be searched.</param>
/// <returns>An IReadOnlyList collection containing the file objects.
</returns>
async Task<System.Collections.Generic.IReadOnlyList<StorageFile>>
GetLibraryFilesAsync(StorageFolder folder)
{
    var query = folder.CreateFileQuery();
    return await query.GetFilesAsync();
}

private async void CheckAvailabilityOfFilesInPicturesLibrary()
{
    // Determine availability of all files within Pictures library.
    var files = await GetLibraryFilesAsync(KnownFolders.PicturesLibrary);
```

```
    for (int i = 0; i < files.Count; i++)
    {
        StorageFile file = files[i];

        StringBuilder fileInfo = new StringBuilder();
        fileInfo.AppendFormat("{0} (on {1}) is {2}",
                    file.Name,
                    file.Provider.DisplayName,
                    file.IsAvailable ? "available" : "not available");
    }
}
```

Feedback

Was this page helpful? Yes No

Provide product feedback | Get help at Microsoft Q&A

Files and folders in the Music, Pictures, and Videos libraries

Article • 10/20/2022

Add existing folders of music, pictures, or videos to the corresponding libraries. You can also remove folders from libraries, get the list of folders in a library, and discover stored photos, music, and videos.

A library is a virtual collection of folders, which includes a known folder by default plus any other folders the user has added to the library by using your app or one of the built-in apps. For example, the Pictures library includes the Pictures known folder by default. The user can add folders to, or remove them from, the Pictures library by using your app or the built-in Photos app.

Prerequisites

- **Understand async programming for Universal Windows Platform (UWP) apps**

  You can learn how to write asynchronous apps in C# or Visual Basic, see Call asynchronous APIs in C# or Visual Basic. To learn how to write asynchronous apps in C++, see Asynchronous programming in C++.

- **Access permissions to the location**

  In Visual Studio, open the app manifest file in Manifest Designer. On the **Capabilities** page, select the libraries that your app manages.
  - **Music Library**
  - **Pictures Library**
  - **Videos Library**

  To learn more, see File access permissions.

Get a reference to a library

> **Note**
>
> Remember to declare the appropriate capability. See **App capability declarations** for more information.

To get a reference to the user's Music, Pictures, or Video library, call the StorageLibrary.GetLibraryAsync method. Provide the corresponding value from the KnownLibraryId enumeration.

- KnownLibraryId.Music
- KnownLibraryId.Pictures
- KnownLibraryId.Videos

C#

```
var myPictures = await
Windows.Storage.StorageLibrary.GetLibraryAsync(Windows.Storage.KnownLibraryI
d.Pictures);
```

Get the list of folders in a library

To get the list of folders in a library, get the value of the StorageLibrary.Folders property.

C#

```
using Windows.Foundation.Collections;
IObservableVector<Windows.Storage.StorageFolder> myPictureFolders =
myPictures.Folders;
```

Get the folder in a library where new files are saved by default

To get the folder in a library where new files are saved by default, get the value of the StorageLibrary.SaveFolder property.

C#

```
Windows.Storage.StorageFolder savePicturesFolder = myPictures.SaveFolder;
```

Add an existing folder to a library

To add a folder to a library, you call the StorageLibrary.RequestAddFolderAsync. Taking the Pictures Library as an example, calling this method causes a folder picker to be shown to the user with an **Add this folder to Pictures** button. If the user picks a folder